VETRA

Backend Guide

Updated October 2026

This document describes the backend that exists and runs today, in `backend/` at the project root. It is not a build plan. Every route, table, and integration below was read out of the code before it was written down here, and anything that is genuinely not built is called out as such in §8 rather than described as if it works.

These documents sit next to each other:

- **This one** covers the API surface, the data model, and why the backend is shaped the way it is.
- `backend/README.md` covers running it locally and how it is deployed.
- `ADMIN_GUIDE.md` covers operating the live deployment: which services it depends on, how to reach the database directly, and how to create a Super Admin without the console.
- `DOCUMENTATION.md` covers what each frontend page does.
- `DELIVERY_SYSTEM.md` and `PAYMENTS.md` cover the delivery system and money (checkout, payouts, refunds) in depth.

1. What is actually running

The backend is a single Express app running on cPanel shared hosting (Namecheap) through **Setup Node.js App** (Phusion Passenger) at `https://api.vetrahq.org`, talking to the account's own MySQL database on the same server. The frontend is four static apps (`customer/`, `vendor/`, `rider/`, `admin/`) plus a public marketing site and the public tracking page (`track.html`), all plain HTML/CSS/JS with no build step, served from the same cPanel account at `https://vetrahq.org`. One cPanel Git deploy (`.cpanel.yml`) ships both. `api-client.js` at the project root is the shared fetch wrapper every page uses, and it hardcodes `VETRA_API_BASE` to `https://api.vetrahq.org/api`. The previous hosts (Render for the API, Vercel for the frontend, Clever Cloud for the database) are kept only as a short rollback window after the cutover; see `backend/README.md`, "Moving to cPanel".

All fourteen route modules in `backend/src/routes/` are real and mounted in `backend/src/app.js`:

Mount	File
`/api/auth`	`auth.routes.js`
`/api/products`	`products.routes.js`
`/api/orders`	`orders.routes.js`
`/api/admin`	`admin.routes.js`
`/api/reports`	`reports.routes.js`

Mount	File
`/api/vendors`	`vendors.routes.js`
`/api/vendors/:vendorId/reviews`	`reviews.routes.js`
`/api/site-banners`	`site-banners.routes.js`
`/api/uploads`	`uploads.routes.js`
`/api/notifications`	`notifications.routes.js`
`/api/delivery`	`delivery.routes.js` (logic in `utils/delivery.js`, see `DELIVERY_SYSTEM.md`)
`/api/contact`	`contact.routes.js` (the public contact form)
`/api/payments`	`payments.routes.js` (logic in `utils/payments.js`, see `PAYMENTS.md`); the webhook is mounted separately in `app.js`, ahead of `express.json()`
`/api/assistant`	`assistant.routes.js`

`GET /api/health` is defined directly in `app.js`, needs no database or token, and returns `{"ok":true}`. It is the quickest check that Passenger has the app up after a deploy.

Frontend wiring is complete for `customer/`, `vendor/`, `rider/`, and all nine pages of the `admin/` console. The old `admin/assets/data.js` localStorage mock has been deleted; its display-only helpers moved to `admin/assets/format-helpers.js`. The one exception is the AI assistant: `POST /api/assistant/chat` works, but no page in `customer/`, `vendor/`, or `admin/` calls it yet, so it is reachable only by direct request.

---

2. Architecture

```
Browser (static HTML/CSS/JS from ~/vetrahq.org on cPanel:
         public site, track.html, customer/, vendor/, rider/, admin/)
        │  HTTPS, JSON over REST, Authorization: Bearer <JWT>
        ▼
Express app at api.vetrahq.org (cPanel Setup Node.js App / Passenger, ~/vetra-api)
        │
        ├── MySQL on the same cPanel account (mysql2/promise pool, plain SQL, no ORM)
        ├── Cloudinary ........ every uploaded file: avatars, cover photos,
        │                       product images/video, site banners, KYC documents
        ├── cPanel SMTP ....... every transactional email (nodemailer, noreply@vetrahq.org)
        ├── Paystack .......... bank list + NUBAN resolution for vendor payout accounts
        ├── IDCheck.ng ........ NIN / BVN / driver's licence / CAC verification for vendor KYC
        └── Anthropic ......... the AI shopping assistant's reply generation

cPanel cron, every 10 min ── scripts/delivery-cron.js ── same database
        (escrow auto-release, expired rider offers, auto-assign retries)
```

Nothing is ever written to local disk. That rule dates from Render, whose free tier had no persistent disk, and it still holds on cPanel: deploys `rsync --delete` the app folder, and uploads served through a CDN beat files on a shared host. Cloudinary is the object store, and `multer` is configured with `memoryStorage()` so an upload goes buffer to Cloudinary without ever touching the filesystem.

There is no WebSocket layer and no message queue. Everything the API does happens inside a request/response cycle. The one scheduled piece is the delivery sweep, a plain cron job (`backend/scripts/delivery-cron.js`) that runs outside the app every 10 minutes and talks to the database directly. The rider app, the buyer's delivery panel, and the dispatch console poll every 20 to 30 seconds instead of receiving pushes.

Connection limits. `backend/src/db.js` defaults `DB_CONNECTION_LIMIT` to `20`, chosen for Namecheap shared hosting's `maxEntryProc` (Entry Processes) cap. That cap covers the whole account, so the Node app, its database connections, and the cron job all share it; `~/vetra-api/.env` sets the limit to `10` to leave headroom. Exceeding the account's cap shows up as `ER_USER_LIMIT_REACHED` or hanging requests under load rather than as a slow query. (On Render, `render.yaml` pinned it to `5`, Clever Cloud's per-user cap; that file goes when the rollback window ends.)

Sessions. A JWT in an `Authorization: Bearer` header, not a server-side session cookie. `backend/src/utils/jwt.js` signs `{ id, role, adminRole, sessionVersion }` and expires per `JWT_EXPIRES_IN` (default `7d`). The token is not the whole story though, because `backend/src/middleware/auth.js` hits the database on every authenticated request and re-checks four things against the live `users` row:

1. The account still exists.
2. Its `status` is not `suspended` or `deleted`, so a suspension takes effect on the next request rather than whenever the token happens to expire.
3. The token's `sessionVersion` still matches `users.session_version`, so incrementing that column revokes every token already issued for the account.
4. `users.last_activity_at` is within `SESSION_IDLE_TIMEOUT_MINUTES` (default 30). Past that, the request gets a 401 even with a structurally valid token.

`req.user.role` and `req.user.adminRole` are taken from the fresh database row, not from the token payload, so a role change also takes effect immediately. The middleware then stamps `last_activity_at = NOW()`. The cost is one extra `SELECT` and one `UPDATE` per authenticated request, paid deliberately: without it, "suspend this account" would mean "suspend this account in up to seven days."

`session_version` is incremented on every sign-in, password change, password reset, self-deactivate, self-delete, admin-initiated suspend/reactivate/reject, admin-initiated email change, and admin role change.

---

3. Data model

MySQL/MariaDB, 26 tables. `backend/migrations/*.sql` is the source of truth; this section explains the reasoning the SQL files do not always spell out. Migrations run in filename order and are tracked in `schema_migrations`, so `npm run migrate` is safe to re-run.

Conventions used throughout:

- **Primary keys** are `CHAR(36)` UUIDs generated by `backend/src/utils/id.js`, which hand-builds a **UUIDv7** rather than calling `crypto.randomUUID()` (a v4). The leading 48 bits are a millisecond timestamp, so new rows append to the end of InnoDB's clustered index instead of landing at random points and forcing page splits. On-disk format is identical to a v4, so nothing else had to change.
- **Money** is an `INT` count of **kobo** (1 naira = 100 kobo), never a float or decimal naira value. Conversion happens only at the UI boundary, in `api-client.js`'s `formatNaira()` / `nairaToKobo()`. A raw database value that looks 100x too large is correct.
- **Arrays** are `JSON` columns (`products.images`, `products.keywords`, `report_evidence.attachment_urls`), since MySQL has no native array type.
- **Timestamps** are real `DATETIME` values returned as JavaScript `Date` objects. `dateStrings` is deliberately off on the pool; see the note in `db.js` for the WAT timezone bug that turning it on caused.

`users`

One table for buyers, vendors, riders, and admins, distinguished by `role`. Four separate tables would mean four copies of auth, password reset, and session logic for what is one concept: an account that can sign in. Role-specific columns sit `NULL` on rows that do not need them.

Column	Notes
`id`	UUIDv7, `CHAR(36)`.
`role`	`buyer` | `vendor` | `rider` | `admin`.
`name`	Display name, required.
`first_name`, `middle_name`, `last_name`	Added in `012_vendor_identity_names.sql`. Legal name parts, required for vendor signup, and what KYC verifies against: they are sent to IDCheck.ng with a NIN or BVN, and compared field by field with the name on a driver's licence record. Vendors cannot edit them. `name` stays the display-name field for everything else.
`email`	Unique **per role** (`uniq_email_role`). The same address can exist once as a buyer and once as a vendor.
`phone`, `address`	Required by the signup routes, nullable on the column.
`state`	One of Nigeria's 36 states or the FCT, validated against `backend/src/utils/nigerianStates.js`. Required on every buyer/vendor signup path including Google. For a rider it is their dispatch zone, changed only through `PATCH /api/delivery/rider/zone` (refused while they hold a job), never through `PATCH /api/auth/me`. Nullable on the column because admin rows have none.
`password_hash`	bcrypt, 10 rounds, via `bcryptjs`. Never reversible.
`status`	`active` | `suspended` | `pending` | `rejected` | `deleted`.
`signup_method`	`email` or `google`.
`avatar_url`, `store_cover_url`	Cloudinary URLs.
`store_name`, `store_category`, `store_description`	Vendor-only.
`rider_vehicle`, `rider_plate`, `rider_on_duty`	Rider-only, added in `015_delivery_system.sql`. Vehicle is `bicycle` | `motorbike` | `car` | `van`. Only on-duty riders are offered jobs.
`admin_role`	`Super Admin` | `Moderator` | `Support`. Admin-only, and deliberately separate from `role`, which only says "this is an admin". Every `requireAdminRole(...)` check reads this.
`kyc_email_alerts_enabled`	Per-admin opt-out of the KYC notification email. Defaults true.
`two_factor_enabled`	Email one-time-code sign-in. Settable via `PATCH /api/auth/me` by vendors and admins only.

Column	Notes
`payout_bank_name`, `payout_bank_code`	Vendor-only. `payout_bank_code` is Paystack's code, needed to re-resolve the account or eventually call Paystack's Transfer API.
`payout_account_name`	Vendor-only, but never vendor-typed. Resolved from Paystack.
`payout_account_number_enc`	The NUBAN, AES-256-GCM encrypted (`backend/src/utils/encryption.js`). Never returned in full by any route.
`last_login_at`, `last_login_ip`	Most recent successful sign-in.
`last_activity_at`	Updated by the auth middleware, drives the idle timeout.
`session_version`	Incrementing this revokes every token already issued for the account.
`password_changed_at`	Set only by a real `PATCH /api/auth/password`. `NULL` means never changed since signup, which the UI shows honestly rather than faking a date.
`created_at`	

Indexes worth knowing: `uniq_email_role (email, role)` and `idx_users_role_status (role, status)`. The second exists because nearly every customer-facing page filters on exactly `role = 'vendor' AND status = 'active'`, which had no indexed path before it.

What `status = 'deleted'` means. Row deletion is not possible without breaking order, review, and report rows that legitimately belong to someone else. `POST /api/auth/delete-account` instead scrubs personal fields (`name` becomes `Deleted User`, `email` becomes a unique `deleted-<id>@vetra.deleted` placeholder, `password_hash` is replaced with a random unusable one, phone/address/avatar/store/payout fields are cleared), sets `status = 'deleted'`, and for a vendor delists every product in the same transaction. The email placeholder also frees the original address: `POST /api/auth/signup` detects a `deleted` row on the same `(email, role)` and renames it out of the way so the address can be reused.

`products`

`id`, `vendor_id`, `name`, `category`, `color`, `storage`, `price` (kobo), `stock_quantity`, `description`, `keywords` (JSON, max 5), `images` (JSON), `video_url`, `status`, `description_embedding` (JSON, unused so far), `created_at`, `updated_at`.

- `color` / `storage` are plain free-text descriptive fields, not a variant system. A vendor listing a phone in several colours writes "Black, Silver" on one listing rather than creating separate SKUs. There is no per-combination price or stock.

- `keywords` are up to 5 vendor-supplied search terms, enforced by `normalizeKeywords()` in `products.routes.js` both on create and on edit. They exist so a listing surfaces for a term that is true of the product but appears in neither its name nor its description.
- `status` (`active` / `out_of_stock` / `removed`) is derived, never set directly by a client. Checkout's stock decrement flips a product to `out_of_stock` when its stock reaches zero; a vendor editing `stockQuantity` to 0 does the same, and editing it back above 0 flips it back. `removed` is a soft delete, set by the vendor's own `DELETE` or by an admin removing a listing, and it is terminal: a `removed` listing cannot be edited back to life.
- Indexes: `idx_products_vendor`, `idx_products_status`, `idx_products_status_category`, and a `FULLTEXT` index `idx_products_search (name, description)`. The FULLTEXT index exists because the old `name LIKE '%text%'` search had a leading wildcard, which no B-tree index can serve.
- `description_embedding` is provisioned for a future semantic search. Nothing writes to it today.

`orders` and `order_items`

An order belongs to one vendor. A multi-vendor cart is paid in one Paystack payment, and `utils/payments.js` creates one order per vendor once that payment is confirmed; see `PAYMENTS.md` §2.

`orders`: `id`, `tracking_code`, `buyer_id` (nullable for guest checkout), `guest_name` / `guest_email` / `guest_phone`, `vendor_id`, `delivery_method`, `delivery_address`, `status`, `carrier`, `tracking_number`, `total` (kobo), `escrow_status`, `escrow_released_at`, `shipped_at`, `out_for_delivery_at`, `delivered_at`, `cancelled_at`, `created_at`, `idempotency_key`.

- `status`: `pending` → `processing` → `shipped` → `out_for_delivery` → `completed`, or `cancelled`. The extra granularity beyond pending/processing/completed is what backs the delivery timeline on `customer/orders.html`.
- `tracking_code` is a VETRA-generated customer-facing code, `"VTA-"` plus 8 characters from a Crockford-style base32 alphabet with no `0` / `O` / `1` / `I`, so it survives being read aloud or typed back in. It is the order's one public ID: buyers, vendors, riders, admins, emails, notifications and the activity log all show it, via `orderRef()` in `utils/id.js` and `formatOrderRef()` in `api-client.js`. It used to sit beside a second reference, `"VTR-"` plus the id's first 8 hex characters, which stopped being unique once ids became time-ordered UUIDv7s (orders within the same ~65 seconds shared one). Orders placed before tracking codes existed have no `tracking_code` and still show that old `VTR-` form, which was unique for their random ids. Activity-log entries written before the switch keep the `VTR-` text they were written with.
- `idempotency_key` belonged to the old one-request-per-order checkout and is no longer written: idempotency now lives on `payments.idempotency_key`. `payment_id` links each order to the payment that paid for it (NULL for orders from before payments), with `commission_percent` (snapshotted at creation), `commission_amount` (set when escrow releases) and `refunded_amount`.
- `escrow_status` (`held` / `released` / `refunded`) tracks real money for paid orders. `released` is set when the buyer taps "I received it", or by the delivery sweep once `platform_settings.auto_confirm_hours` (48 by default) have passed since `delivered_at` with no open report on the order, and releasing queues the vendor's payout. `refunded` is set when an order is cancelled, a parcel is returned, or a Super Admin refunds it in full, and the buyer is refunded to their card. See `DELIVERY_SYSTEM.md` and `PAYMENTS.md`.

- Delivery columns: `delivery_fee` (kobo, on top of `total`, which stays the vendor's goods total), `delivery_state`, `delivery_phone`, `delivery_notes`, `delivery_code` (the buyer's 6-digit proof of receipt, never returned to vendor, rider, or admin routes), `delivery_code_attempts`, `buyer_confirmed_at`. Delivery-method orders also get one `deliveries` row; see `DELIVERY_SYSTEM.md` for that table, `delivery_events`, and `delivery_rates`.

`order_items`: `id`, `order_id`, `product_id`, `quantity`, `price_at_purchase` (a snapshot, not a live join to `products.price`), `status` (`fulfilled` / `unavailable`), `unavailable_reason`.

The per-item `status` lets a vendor drop one line item from a multi-item order instead of cancelling everything. `orders.total` is recomputed from the still-fulfilled items, and if none remain the order itself cancels.

`reviews`

`id`, `vendor_id`, `buyer_id`, `order_id`, `rating`, `review_text`, `created_at`. `order_id` is `NOT NULL`, which is what makes "verified purchase" real rather than a label. `UNIQUE KEY uniq_review_per_order (order_id)` enforces one review per order at the database level, and `CHECK (rating BETWEEN 1 AND 5)` enforces the range there too. The route additionally verifies server-side that a `completed` order exists linking that buyer and vendor.

`reports` and `report_evidence`

`reports`: `id`, `type` (`customer` / `vendor` / `product`), `target_id`, `order_id`, `reporter` (display text), `reporter_user_id`, `reason`, `status` (`open` / `resolved` / `dismissed`), `attended_by_user_id`, `attended_at`, `created_at`. In practice only `type = 'vendor'` rows are ever created today, by a buyer reporting a vendor over a specific order or by the auto-flag scan.

`report_evidence`: `id`, `report_id`, `vendor_user_id`, `response_text`, `attachment_urls` (JSON), `submitted_at`. Append-only; submitting evidence never changes the parent report's status.

`activity_log`

`id`, `type` (`account` / `vendor` / `report` / `order` / `login` / `delivery`), `message`, `actor_user_id` (nullable, `NULL` for system and non-staff events), `target_type`, `target_id`, `created_at`.

Written exclusively by `logActivity()` in `backend/src/utils/activityLog.js`, called from the route handler that performed the mutation. Never client-supplied. The helper is deliberately best-effort: every call site does its real primary write first, and a failure to log is caught and logged to the console rather than turned into a 500, because a 500 after a successful write invites a retry that duplicates the action.

`actor_user_id` is `ON DELETE SET NULL`. Without that, an admin who had ever signed in could not be removed from the team at all, because their own login rows blocked the delete.

`vendor_kyc`

One row per vendor, `vendor_id` as the primary key, created lazily on first submission. A separate table rather than columns on `users` because this is a review workflow with its own lifecycle, not a static profile field.

`status` (`not_submitted` / `pending` / `manual_review` / `verified` / `rejected`), `cac_number`, `identity_type` (`nin` / `bvn` / `drivers_license`; `bvn` added in `016_idcheck_identity_types.sql`), `identity_number_enc` (AES-256-GCM, same key as payout accounts), `identity_provider_status`, `identity_provider_message`, `identity_verified_at`, `cac_provider_status`, `cac_provider_message`, `cac_verified_at`, `id_document_url`, `cac_document_url`, `submitted_at`, `reviewed_at`, `reviewed_by_user_id`, `rejection_reason`.

The provider's raw response payload is deliberately not persisted, since it can contain identity and biometric data this app has no reason to keep.

Status lifecycle. A vendor uploads both documents (`POST /me/kyc`), then runs verification (`POST /me/kyc/verify`), which makes two IDCheck.ng calls (`backend/src/utils/idcheck.js`). The identity check depends on the type: a NIN or BVN is sent with the vendor's legal `first_name` / `last_name` (and `middle_name` for a NIN) and IDCheck answers MATCH or MISMATCH; a driver's licence is looked up by number and the date of birth the vendor typed (sent, never stored), and the name on the returned record is compared with the legal name here, after normalisation that strips accents, punctuation, and case. The CAC check looks up the RC or BN number. If both pass, the status goes straight to `verified` with no human involved. If either fails, the provider is unreachable or out of credit, or the number is an `IT` (incorporated trustee) number, which IDCheck's CAC lookup does not cover, the status becomes `manual_review` and every admin is notified. An admin then resolves it to `verified` or `rejected` via `PATCH /api/admin/vendors/:id/kyc`, which accepts a submission in `pending` or `manual_review` state only. `rejection_reason` is cleared unconditionally on a verify, so an old reason cannot resurface after a later approval. Rows verified before October 2026 were checked by CheckID.ng, the previous provider.

`admin_invites`

`id`, `name`, `email`, `admin_role`, `invited_by_user_id` (`ON DELETE SET NULL`), `verification_code_hash` (bcrypt), `expires_at` (15 minutes), `status` (`pending` / `verified` / `cancelled`), `created_at`.

`notifications`

`id`, `user_id`, `type` (`order` / `kyc` / `vendor_status` / `account` / `report` / `low_stock` / `delivery`), `title`, `message`, `link` (a relative frontend path), `read_at`, `created_at`. `ON DELETE CASCADE` on `user_id`.

Written by `notify()` in `backend/src/utils/notify.js`, same best-effort pattern as `logActivity()`. Not every mutation produces one, only changes the recipient would want to know about without checking.

`password_reset_tokens`

`id`, `user_id`, `token_hash` (SHA-256 of a 24-byte random token), `expires_at` (1 hour), `used_at`, `created_at`. A fast hash is fine here, unlike for a password or a 6-digit code, because the token itself already carries far more entropy than either.

`two_factor_codes`

`id`, `user_id`, `code_hash` (SHA-256), `expires_at` (10 minutes), `consumed_at`, `attempts`, `created_at`. Issuing a new code marks any earlier unconsumed code for that user consumed, so at most one is ever valid. Old rows are kept rather than deleted, both as an audit trail and so a wrong-guess counter cannot be reset by requesting a fresh code.

`pending_email_changes`

`id`, `user_id`, `new_email`, `code_hash` (SHA-256), `requested_by_admin_id`, `attempts`, `expires_at` (10 minutes), `consumed_at`, `created_at`. Same shape as `two_factor_codes`, for the admin-initiated email change flow in §5.

`login_ip_history`

`id`, `user_id`, `ip_address`, `occurred_at`. One row per successful sign-in, for admin audit review. `users.last_login_ip` holds just the most recent one for quick display.

`vendor_payout_account_history`

`id`, `vendor_id`, `bank_name`, `bank_code`, `account_name`, `masked_account_number`, `linked_at`, `unlinked_at`. Only the masked number is recorded here. The encrypted real number stays on `users` and is never exposed to admins through any route.

`site_banners`

`id`, `image_url`, `alt_text`, `display_order`, `created_at`. Deliberately picture-only: no title, subtitle, or link target, because the carousel this backs is picture-only by design.

`platform_settings`

One row, `id` always `1`. Six boolean toggles behind `admin/settings.html`'s Platform Controls card, plus five delivery settings (`free_delivery_threshold`, `rider_payout_percent`, `auto_assign_riders`, `auto_confirm_hours`, `max_delivery_attempts`) edited from `admin/deliveries.html` and documented in `DELIVERY_SYSTEM.md`. The toggles:

Column	What it actually gates
`guest_checkout_enabled`	`POST /api/orders` rejects an unauthenticated order with a 403 while off.
`vendor_approval_required`	New vendor signups (both `/signup` and `/google`) start `pending` while on, `active` while off.

Column	What it actually gates
`vendor_verification_required`	Blocks `PATCH /api/admin/vendors/:id/status` from moving a `pending` vendor to `active` unless their KYC is `verified`, and gates `POST /api/products` the same way. Only the `pending → active` transition is gated; reactivating a previously suspended vendor is not, since they cleared the bar once already.
`auto_flag_listings`	`POST` / `PATCH /api/products` scan name and description against `RESTRICTED_LISTING_KEYWORDS`; a match files a real report rather than blocking the listing.
`maintenance_mode`	Returns 503 from `POST /api/auth/signup` and `POST /api/orders`. Browsing and sign-in stay up; this is a pause on new state, not an outage.
`kyc_email_alerts_enabled`	Super Admin master switch over every individual admin's own `kyc_email_alerts_enabled`. Off here means no admin gets the KYC email regardless of their own setting. The in-app notification always fires.

`delivery_rates`, `deliveries`, `delivery_events`

Added in `015_delivery_system.sql`. `delivery_rates` holds one row per state (intra- and inter-state fee in kobo, ETA range, active flag). `deliveries` is one row per delivery-method order: mode (`vetra` or `self`), status, rider, pickup code, address snapshots, fee charged, rider payout, failed attempts, proof photo, last rider position, and a timestamp per step. `delivery_events` is its append-only timeline. `deliveries.order_id` and `delivery_events.delivery_id` cascade on delete; `deliveries.rider_id` and `delivery_events.actor_user_id` are `SET NULL`. Full detail in `DELIVERY_SYSTEM.md`.

`payments`, `payouts`, `refunds`

Added in `017_payments.sql`. `payments` is one row per checkout, holding the cart snapshot until Paystack confirms the charge; `payouts` one per vendor or rider payout, unique per job so nothing is paid twice; `refunds` one per refund. Full detail in `PAYMENTS.md` §8. `users.paystack_recipient_code` caches the Paystack transfer recipient, and `platform_settings` gains `commission_percent` and `auto_payouts_enabled`.

`contact_messages`

Added in `018_contact_messages.sql`: one row per message from `contact.html` (sender type, name, email, topic, order ID or business name, message, status `new` / `replied` / `closed`, the reply, who replied and when, the sender's IP). Stored raw and escaped wherever it's displayed.

`schema_migrations`

`filename`, `applied_at`. Created by `backend/scripts/migrate.js` on first run, one row per applied file.

---

4. Authentication and the four account types

Sign-up and sign-in

Buyers and vendors share `POST /api/auth/signup` and `POST /api/auth/signin` with a `role` field. Riders sign in through the same `/signin` with `role: "rider"` but can never sign up: an admin creates the account from the Deliveries console and the rider sets a password from an emailed 72-hour link (the password-reset token flow, reused). Admins have their own `POST /api/auth/admin-signin`, because admin accounts are never self-service signups: they are created by an invite flow or, in an emergency, directly against the database (see `ADMIN_GUIDE.md` §3).

Passwords are hashed with `bcryptjs` at 10 rounds. `bcryptjs` is the pure-JS implementation rather than native `bcrypt` specifically because it needs no compiler toolchain, which mattered when shared hosting was the target and costs nothing now.

Sign-in rejects a `suspended` account with a 403. The auth middleware separately rejects both `suspended` and `deleted` accounts on every authenticated request.

Google Sign-In

`POST /api/auth/google` is a single combined signup-or-signin. The frontend (`google-signin.js`) uses Google Identity Services' OAuth2 **token client** (`google.accounts.oauth2.initTokenClient`), not the ID-token/One Tap credential flow, because the token client reliably opens a popup from a click on this site's own custom-styled button, where the credential flow needs Google's own rendered button.

`backend/src/utils/googleAuth.js` verifies the access token in two steps, needing only `GOOGLE_CLIENT_ID` and never a client secret:

1. `https://oauth2.googleapis.com/tokeninfo` confirms the token's `aud` is this app's client id. Skipping this would let a valid Google token issued to an unrelated app be replayed here to claim that user's email.
2. `https://www.googleapis.com/oauth2/v3/userinfo` fetches `{ email, name, picture }`, and the route rejects an unverified Google email.

A new account gets `signup_method = 'google'` and a random unusable password hash, which satisfies the `NOT NULL` column without making it nullable for one signup path. The response includes `needsProfileCompletion`, true whenever `phone`, `address`, `state`, or (for a vendor) `store_name` / `store_category` are missing, which is always true for a brand new Google account since Google supplies none of them.

Two-factor authentication

Email one-time code, not TOTP or SMS, because this deployment already has a working transactional email path and no other second-factor infrastructure. Toggled through `PATCH /api/auth/me`'s `twoFactorEnabled`, which `fieldMap` exposes only to vendor and admin sessions.

When it is on, `/signin` and `/admin-signin` stop short of issuing a token, email a 6-digit code, and return `{ twoFactorRequired: true, challengeToken }`. The challenge token is a 10-minute JWT (`purpose: "2fa"`, bound to `session_version`) proving the password step passed; `/2fa/verify` and `/2fa/resend` require it, it can't be used as a session token, and it dies as soon as sign-in completes. Before this, both endpoints took a bare `userId`, so anyone who knew an account id (vendor ids are public in store URLs) could request and guess codes without the password. No `last_login_at` update, no login activity row, no IP recorded until `POST /api/auth/2fa/verify` actually completes the sign-in. The per-code attempt cap is 5, which is the real brute-force defence for a 1-in-1,000,000 code; the per-IP rate limiter is defence in depth on top of it. Both `signin.html` and `admin/login.html` implement the second step.

Admin roles

Three values in `users.admin_role`, enforced server-side by `requireAdminRole(...)`, an allow-list:

Action	Super Admin	Moderator	Support
View customers, vendors, reports, activity, settings	yes	yes	yes
Reset a customer's or vendor's password	yes	yes	yes (a help-desk task, not a judgement call)
Suspend / reactivate a customer	yes	yes	no
Approve / suspend / reject a vendor	yes	yes	no
Review KYC (verify / reject)	yes	yes	no
Resolve / dismiss a report	yes	yes	no
Remove a vendor's listing	yes	yes	no
Change another account's email or phone	yes	yes	no
View a vendor's payout account and history	yes	yes	no
View another admin's activity (`?adminId=`)	yes	no (own actions only)	no (own actions only)
Manage the admin team (invite, verify, cancel, promote, remove)	yes	no	no
Delete a customer or vendor account	yes	no	no
Change platform settings and site banners	yes	no	no
Prune the activity log	yes	no	no
See the Paystack account balance	yes	no	no

The reasoning, so it does not have to be re-derived: Support can see almost everything and change almost nothing, because Support answers "what is going on with this account." Moderator is the day-to-day trust and safety role: every judgement call about a specific account, report, or submission, but nothing that reconfigures the platform or changes who else has admin access. Super Admin is the only role that can change what the platform is, or who can act as an admin, both categories where a mistake or a compromised account is much harder to undo than a wrong suspend.

`admin/assets/session.js`'s `canModerate()` / `isSuperAdmin()` mirror this in the UI, so a Support admin does not see a button for an action their role cannot take. That is a convenience, not the boundary. The boundary is `requireAdminRole`.

Account lifecycle

`PATCH /api/auth/deactivate` sets the caller's own `status = 'suspended'`, the same end state an admin-initiated suspension produces, just a different actor. Reversible by contacting support, which matches the UI copy.

`POST /api/auth/delete-account` is terminal and is the scrub-and-mark flow described under `users` in §3.

An admin can also delete a customer or vendor account (`DELETE /api/admin/customers/:id`, `DELETE /api/admin/vendors/:id`, both Super Admin only), which performs the same scrub.

---

5. API reference

Two shared pieces wrap every route, so they are not repeated per endpoint:

- `asyncHandler` (`backend/src/utils/asyncHandler.js`) wraps each handler so a rejected promise reaches the error middleware instead of crashing the process. Express 4 does not do this for async functions.
- `errorHandler` (`backend/src/middleware/errorHandler.js`) is the last middleware. A MySQL `ER_DUP_ENTRY` becomes `409 {"error": "That already exists."}`. Otherwise it reads `err.status` or `err.statusCode` and returns that status with the error's own message, except for 500, which returns a generic `{"error": "Something went wrong."}` with the real error logged server-side only. Reading both `status` and `statusCode` matters: `idcheck.js` and `paystack.js` set the latter, and only reading the former collapsed their intended 503/502/422 into a generic 500.

Auth on a route is one of: **public**, **optionalAuth** (`req.user` is populated if a valid token is present and `undefined` otherwise), or **required**, meaning `requireAuth` then `requireRole(...)` and, on admin routes, `requireAdminRole(...)`. Error shapes: `401 {"error": "Missing bearer token."}`, `401 {"error": "Invalid or expired token."}`, `401 {"error": "This account is no longer active."}`, `401 {"error": "Session revoked. Please sign in again."}`, `401 {"error": "Session expired due to inactivity."}`, `403 {"error": "Not allowed for this account type."}`, `403 {"error": "Not allowed for this admin role."}`.

Every list route caps at `LIMIT 200` (100 for notifications). That is a safety net against an unbounded result set, not real pagination, which does not exist yet.

`/api/auth` (`auth.routes.js`)

`POST /signup` — public, `signupLimiter`. Body: `{ role, name, email, password, phone, address, state, firstName?, middleName?, lastName?, storeName?, storeCategory?, confirmOtherRole? }`. `role` must be `buyer` or `vendor`. `name`, `email`, `password`, `phone`, and `address` are all required. `state` must be in `NIGERIAN_STATES`. A vendor signup additionally requires `storeName`, `storeCategory`, `firstName`, and `lastName`. `201`: `{ token, user: { id, role, name, email, status } }`, where `status` is `pending` for a new vendor while `vendor_approval_required` is on, `active` otherwise. `400` on any missing or invalid field. `409` if that `(email, role)` pair is already taken, with a message naming the role. `409 { error, existingRole }` if the email exists under the *other* role and `confirmOtherRole` was not sent, which lets the frontend ask before creating a second account for the same person. `503` while `maintenance_mode` is on. Side effects for a vendor: an `activity_log` row with no actor, a welcome email, and, only when KYC genuinely gates their visibility (`status === 'pending'` **and** `vendor_verification_required` on), an in-app notification saying so. When it does not gate visibility, the email encourages KYC for the Verified badge instead of asserting something false.

`POST /google` — public, `googleAuthLimiter`. Body: `{ accessToken, role }`. `200`: `{ token, user: { id, role, name, email, status, avatarUrl }, needsProfileCompletion }`. `400` missing field. `401 {"error": "Couldn't verify Google sign-in."}` for a bad, expired, wrong-audience, or unverified-email token. `403` for a returning suspended account.

`POST /signin` — public, `signinLimiter`. Body: `{ role, email, password }`. `200`: `{ token, user: { id, role, name, email, status, avatarUrl } }`, or `{ twoFactorRequired: true, challengeToken }` when 2FA is on. `400` bad role. `401 {"error": "Incorrect email or password."}`, deliberately not distinguishing which half was wrong. `403` suspended. Side effects on a completed sign-in: `last_login_at`, `last_activity_at`, `session_version + 1`, a login activity row, and a `login_ip_history` row.

`POST /admin-signin` — public, `adminSigninLimiter` (5 per 15 min, the tightest limit in the app, since a compromised admin reaches every other account). Body: `{ email, password }`. No `role`, since this route only looks at `role = 'admin'` rows. `200`: `{ token, user: { id, name, email, adminRole, avatarUrl } }`, or the same `twoFactorRequired` shape.

`POST /2fa/verify` — public, `twoFactorVerifyLimiter`. Body: `{ challengeToken, code }`. `401` if the challenge is missing, expired, forged, or already used. No `role` field: the pending user's own row decides whether the response is the buyer/vendor shape or the admin shape. `200`: exactly what the relevant sign-in route would have returned. `400` missing field, or no 2FA-enabled account for that id. `401` on `Incorrect code.` (increments that code row's `attempts`), `That code has expired. Request a new one.`, or `Too many incorrect attempts. Request a new code.` `403` suspended.

`POST /2fa/resend` — public, `twoFactorResendLimiter` (5 per 15 min, tighter than verify, because this is the email-bombing vector). Body: `{ challengeToken }`. `200`: `{ ok: true }`. Invalidates whatever code was pending.

`GET /me` — `requireAuth`, any role. Returns `id, role, name, email, phone, address, state, avatar_url, store_name, store_category, store_description, store_cover_url, admin_role, kyc_email_alerts_enabled, two_factor_enabled, created_at, password_changed_at`. Sign-in only returns what is needed at that moment, so any page that displays or edits the full profile needs this.

`PATCH /me` — `requireAuth`, any role. The single endpoint behind every per-field pencil-edit save site-wide, and where a freshly uploaded avatar or cover URL from `POST /api/uploads` actually gets attached to the account. Body: any subset of `{ name, email, phone, address, state, avatarUrl }`. A vendor session additionally accepts `{ storeName, storeCategory, storeDescription, storeCoverUrl, twoFactorEnabled }`; an admin session additionally accepts `{ kycEmailAlertsEnabled, twoFactorEnabled }`. Keys not in the role's `fieldMap` are silently ignored rather than rejected. `200`: the updated row. `400` if `state` is not a valid Nigerian state, or if the body contains none of the recognised keys. `409` if the new email collides with `uniq_email_role`, via the shared duplicate-key handler.

`PATCH /password` — `requireAuth`, any role. Body: `{ currentPassword, newPassword }`. `currentPassword` is required and checked with `bcrypt.compare` regardless of what any form collects, because a stolen token would otherwise be enough to lock the real owner out. `400` missing field or a new password under 8 characters. `401 {"error": "Current password is incorrect."}`. Side effects: `password_changed_at = NOW()`, `session_version + 1` (so every other session is signed out, which is the point of this endpoint), and an `account` activity row.

`POST /reset-password` — public, `resetPasswordLimiter`. Redeems the link an admin-triggered reset emailed out. Body: `{ token, newPassword }`. `400` for a missing field, a short password, or `This reset link is invalid or has expired.` (unknown, already used, or past its 1-hour expiry, all one message). Side effects: updates the hash, marks the token used, bumps `session_version`, writes an activity row.

`PATCH /deactivate` — `requireAuth`, any role, no body. Sets own `status = 'suspended'`, bumps `session_version`, writes an activity row.

`POST /delete-account` — `requireAuth`, any role, no body. The transactional scrub described in §3. The activity row is written with `actorUserId: null`, since the account no longer identifies as its old self.

`/api/uploads` (`uploads.routes.js`)

`POST /` — `requireAuth`, any role, `uploadLimiter` (30 per 15 min, because each request holds a file in memory and consumes Cloudinary bandwidth). `multipart/form-data`: a `file` field and an optional `folder` field (free text, sanitised to `[a-z0-9_-]`, capped at 40 characters, defaulting to `misc`). Accepted types: JPEG, PNG, WEBP, GIF, PDF, MP4, WEBM, QuickTime. Max 20MB, enforced by multer. `MAX_IMAGE_BYTES` and `MAX_VIDEO_BYTES` are separate named constants even though both are 20MB today, so they can diverge later. `201`: `{ url, publicId }`. `400` for a missing file, a disallowed MIME type, or an oversized file.

Two behaviours matter here. First, `folder: "kyc"` uploads to Cloudinary as `type: "authenticated"` and returns a **signed** URL, so a KYC document is not a plain public CDN link the way a product photo is. Second, `folder: "kyc"` is rejected with a 403 for any non-vendor session.

The route never persists anything itself. Upload, get a URL, then `PATCH /api/auth/me`, `POST /api/vendors/me/kyc`, `POST /api/site-banners`, or a product create/edit with that URL.

`/api/products` (`products.routes.js`)

`GET /` — public. Query: `?vendor=<id>`, `?category=<name>`, `?q=<text>`, `?includeOutOfStock=1` (only meaningful alongside `?vendor=`). `200`: array of `{ id, vendor_id, vendor_name, name, category, color, storage, price, stock_quantity, description, keywords, images, video_url, status, created_at, vendor_kyc_verified, sales_count }`, newest first. `sales_count` is a live aggregate (`SUM(order_items.quantity)` over that product's `completed` orders), not a stored counter. It drives the "Hot" badge in `customer/assets/products.js`. `vendor_kyc_verified` rides along so a product card can show the Verified tick without a second request. Without `?vendor=`, the owning vendor must also be `status = 'active'`, so a pending or suspended vendor's products are not publicly browsable. With `?vendor=`, that check is skipped on purpose, so a still-pending vendor can manage their own catalogue. `?q=` matching runs `MATCH(name, description) AGAINST(... IN BOOLEAN MODE)` with each word of 3+ characters as a required prefix (`+word*`). Shorter words are dropped from the fulltext attempt because this managed database's `innodb_ft_min_token_size` is 3 and cannot be lowered without a server restart. Both the fulltext attempt and its `LIKE` fallback also OR in `JSON_SEARCH(p.keywords, 'one', ?)`, since MySQL cannot FULLTEXT-index a JSON column.

`GET /:id` — public. `SELECT p.*` plus `vendor_name`, `vendor_status`, `vendor_kyc_verified`. Requires `p.status = 'active' AND v.status = 'active'`. `404 {"error": "Product not found."}` covers both "does not exist" and "exists but is hidden", same reasoning as the sign-in errors.

Everything below requires `requireAuth` + `requireRole("vendor")`, applied once via `router.use()`.

`POST /` Body: `{ name, category, price, color?, storage?, stockQuantity?, description?, keywords?, images?, videoUrl? }`. `price` must be a positive whole number of kobo; `stockQuantity` must be a non-negative whole number. When `vendor_verification_required` is on, the vendor's own `vendor_kyc.status` must be `verified` **and** both document URLs must be present, otherwise `403`. When the setting is off, the check is skipped, matching what the settings page itself promises. `201`: `{ id }`. Inserted as `out_of_stock` when `stockQuantity` is 0, `active` otherwise. `vendor_id` always comes from the token, never from the body. Side effects: the auto-flag scan, and a low-stock alert if the listing starts at or below `LOW_STOCK_THRESHOLD`.

`PATCH /:id` — ownership-checked. Body: any subset of `{ name, category, color, storage, price, stockQuantity, description, videoUrl, images, keywords }`. `status` is not directly settable; it is derived from `stockQuantity` as described in §3. `200`: `{ ok: true }`. `404` not found. `403` `You don't own this product.` or `This listing was removed by an administrator.` `400` on an invalid price/stock value or an empty body. Side effects: the auto-flag scan re-runs on every edit (simpler than tracking which fields moved, and cheap), and `alertIfLowStock()` fires when stock crosses the threshold downward.

`DELETE /:id` — ownership-checked, soft delete to `status = 'removed'` so `order_items` foreign keys stay intact.

`/api/vendors` (`vendors.routes.js`)

`/me/...` routes are declared before `/:id` so `me` is never swallowed by the param route.

The four payout-account routes below come from `utils/payoutAccount.js`, shared with the rider app (mounted there at `/api/delivery/rider/payout-account`). Saving an account also clears the cached Paystack recipient and sends any payouts that were waiting for one.

`GET /me/payout-account/banks` — vendor. `200`: `[{ name, code }]` from Paystack's Miscellaneous API, cached in memory for 24 hours. A `<datalist>` only ever hands back the typed name, so the code is what the routes below actually need.

`GET /me/payout-account/resolve` — vendor. Query `?accountNumber=<10 digits>&bankCode=<code>`. `200`: `{ accountName, accountNumber }`, the real bank-registered name, used for the form's live "is this you?" preview. Nothing is persisted. `400` missing or non-10-digit input. `422` with Paystack's own message when the pair does not resolve.

`GET /me/payout-account` — vendor. `200`: `{ isSet, bankName, bankCode, maskedAccountNumber, accountName }`, or `isSet: false` with nulls. The mask is derived by decrypting server-side and taking the last four digits; the plaintext number is never serialised into a response.

`PUT /me/payout-account` — vendor. Body: `{ bankName, bankCode, accountNumber }`. There is deliberately no `accountName` field: the name is resolved fresh against Paystack inside this route, so a client cannot submit a name the bank does not have on file. `200`: `{ isSet: true, bankName, bankCode, maskedAccountNumber, accountName }`. `400` missing or malformed input. `422` if the pair does not resolve. Side effects: the number is AES-256-GCM encrypted before the write; any previous history row is stamped `unlinked_at`, and a new `vendor_payout_account_history` row records the masked number. A `PUT` rather than a `PATCH` because a re-save always resubmits the whole account.

`GET /me/kyc` — vendor. `200`: the camelCased `vendor_kyc` row including identity and CAC provider statuses. A vendor who has never submitted gets `status: "not_submitted"` with nulls, not a 404. While `status` is `manual_review`, the provider messages are replaced with a generic "Your submission needs manual review." rather than surfacing raw provider diagnostics to the vendor; the real messages stay visible to admins.

`POST /me/kyc` — vendor. Uploads the documents. Body: `{ cacNumber, idDocumentUrl, cacDocumentUrl }`. Both URLs are validated to be `https://res.cloudinary.com/...` paths containing `/authenticated/`, so a vendor cannot point this at an arbitrary external host or at a plain public upload. `400 {"error": "KYC documents must be uploaded through VETRA."}` otherwise. `201`: `{ status, submittedAt }`. A submission is allowed from `not_submitted`, `rejected`, `manual_review`, or `verified`; only an existing `pending` blocks a replacement. An existing `verified` or `manual_review` status is preserved rather than reset to `pending`. Side effects: every admin gets an in-app notification unconditionally; the email is sent only when both the platform-wide `kyc_email_alerts_enabled` and that admin's own flag are on.

`POST /me/kyc/verify` — vendor, `kycVerifyLimiter` (5 an hour per vendor, since every attempt is a paid lookup). Runs the IDCheck.ng checks described under `vendor_kyc` in §3. Body: `{ identityType: "nin" | "bvn" | "drivers_license", identityNumber, dateOfBirth?, cacNumber }`. A NIN or BVN is 11 digits. A

licence number is 6 to 20 letters, digits, or dashes, and needs `dateOfBirth` (`YYYY-MM-DD`, in the past). The CAC number is `RC`, `BN` or `IT` followed by digits; spaces and dashes are stripped before the lookup. `400` if either document has not been uploaded yet, on a malformed number or date, or when the vendor has no legal first and last name on file. `503` when `IDCHECK_API_KEY` is unset, surfaced as "Identity verification is not configured yet." `200` when both checks pass, including the name match. `422` otherwise, with the per-check status and message. A sandbox (`sk_test_`) key's results carry "(IDCheck.ng sandbox, not a real check)" in their messages. Either way the `vendor_kyc` row is upserted with `status` set to `verified` or `manual_review`, the identity number encrypted, and both provider statuses/messages recorded. A `manual_review` result notifies and optionally emails every admin, and notifies the vendor. Notifications are suppressed when nothing meaningful changed since the last attempt, so repeated retries with identical inputs do not spam anyone.

`GET /` — public. Query `?q=` matches `store_name` with a `LIKE`. `200`: `{ id, store_name, avatar_url, store_cover_url, store_category, status, address, rating, review_count, kyc_verified }` for `status = 'active'` vendors only. `rating` / `review_count` are live `AVG` / `COUNT` over `reviews`, not stored columns. `kyc_verified` is the real KYC outcome, which is a separate thing from being approved to sell: a vendor can be live without ever passing KYC, so the badge has to reflect the former.

`GET /:id` — public. Same fields plus `store_description`, `member_since`, `products_count`, and `orders_count`. `404` if not found or not `active`.

`/api/vendors/:vendorId/reviews` (`reviews.routes.js`, mounted with `mergeParams`)

`GET /` — public. `200`: `{ average, count, reviews: [{ id, rating, review_text, created_at, buyer_name }] }`. `average` is `null` rather than `0` when there are no reviews, so the UI can tell "none yet" from "reviews exist and are bad". Both aggregates come from a separate query over every review, not from the capped list, which would under-report for any vendor past the cap.

`POST /` — `requireAuth` + buyer. Body: `{ rating, text, orderId }`. `400` for a rating outside 1 to 5, empty text, or text over 2000 characters. `403 {"error": "You can only review a vendor after a completed order."}` unless an order exists matching `id = orderId AND buyer_id = <token> AND vendor_id = :vendorId AND status = 'completed'`. A client cannot fake this with an arbitrary `orderId`. `409` via the duplicate-key handler on a second review for the same order.

`/api/orders` (`orders.routes.js`)

`POST /` — retired. Answers `410` "Checkout has moved." Checkout is `POST /api/payments/checkout` (`PAYMENTS.md` §2), and orders are created by `utils/payments.js` once Paystack confirms the payment. The validation it used to do (one vendor per order, each product once, quantities 1 to 1000, active products from an active vendor, enough stock, server-side prices and delivery quote) moved there unchanged, as did the guarded stock decrement, now done when the checkout is created and undone if it's never paid.

`GET /mine` — buyer. Every order for the signed-in buyer, newest first, each with `vendor_name`, `vendor_kyc_verified`, and an `items` summary built by the shared `ORDER_ITEMS_SUBQUERY` (`JSON_ARRAYAGG` of id, productId, name, quantity, price, first image, status, unavailableReason). One query, not a round trip per order.

`GET /vendor` — vendor. Optional `?status=`. Same item summary, plus `buyer_name` and `buyer_phone` via `COALESCE` over the account fields and the guest fields, so the order-detail expand can show a real contact card without a second request.

`PATCH /:id/shipment` — vendor, ownership-checked. Body: `{ status, carrier?, trackingNumber? }`. `400` for a status outside the six. `404` not found. `403 {"error": "This isn't your order."}`. Side effects: stamps the matching timestamp column (`pending` and `processing` stamp nothing). `completed` additionally sets `escrow_status = 'released'` and `escrow_released_at`. Writes an activity row, notifies the buyer for every status except `pending`, and emails for `processing`, `shipped`, `out_for_delivery`, and `completed` only. A guest order has no account to notify in-app but still gets the email.

`PATCH /:id/items/:itemId/unavailable` — vendor, ownership-checked. Body: `{ reason? }`, free text shown back to the buyer. `200`: `{ ok: true, total, orderCancelled }`. `400` if the order is past `processing` ("Can't remove an item once the order has shipped, cancel the whole order instead if it can't be fulfilled."), or if the item is already unavailable. `404` for an unknown order or an item not on it. `403` not this vendor's order. Side effects: marks the item, recomputes `orders.total` from the remaining fulfilled items, and if none remain sets the order `cancelled` with `escrow_status = 'refunded'`. Writes an activity row, notifies the buyer, and emails the adjusted itemisation. This exists rather than "cancel the whole order" because a vendor who cannot fulfil one line of a multi-item order should not have to cancel the rest. It is deliberately about availability, a one-off "turns out I do not have this one", not a structured multi-SKU inventory system.

`buildOrderEmailHtml()` (in `utils/orderEmail.js`, shared with the delivery emails) renders the itemised products (fulfilled ones only), the store name, delivery method and address, the delivery fee and total, the order ID (tracking code), and, for buyer emails only, the delivery code. Every vendor- or user-supplied string going into it is `escapeHtml()`'d first, because these render as HTML in a real inbox and an unescaped `<img onerror=...>` in a product name would otherwise execute there.

`/api/delivery` (`delivery.routes.js`)

Fee quotes, public tracking, the vendor's delivery actions, the rider app's job actions, and the dispatch console, with the state machine itself in `utils/delivery.js`. The full endpoint table, the status flow, and the code rules are in `DELIVERY_SYSTEM.md` rather than repeated here. The two public routes (`GET /quote`, `GET /track/:code`) sit behind `deliveryPublicLimiter`, and every pickup or delivery code entry behind `deliveryCodeLimiter`. `POST /api/orders` runs the same quote logic itself and never trusts the cart's delivery fee.

`/api/payments` (`payments.routes.js`)

Buyer checkout, the return page's status check, and Paystack's webhook. Endpoints, the flow and the money rules are in `PAYMENTS.md`. The admin side (settings, payouts, refunds, the balance) lives in `admin.routes.js`, Super Admin only.

`/api/contact` (`contact.routes.js`)

`POST /`: public, `contactLimiter` (5 an hour per IP). Body `{ senderType: "buyer" | "vendor", name, email, message, topic?, orderRef?, businessName?, website? }`. `website` is a honeypot field the page hides from people: if it's filled, the route answers 201 and stores nothing. `400` for a missing name, a malformed email, or a message under 10 characters. On success: the row is stored, an activity row is written, every active admin gets an in-app notification, and the sender gets a confirmation email.

The admin side is in `admin.routes.js`, open to every admin role, since answering people is help-desk work: `GET /messages?status=`, `POST /messages/:id/reply` (`{ reply }`: emails the sender and marks it replied; answers `502` and saves nothing if the email can't be sent), `PATCH /messages/:id` (`{ status }`). `GET /stats` gains `newMessages`.

`/api/reports` (`reports.routes.js`)

`router.use(requireAuth)` applies to the whole file; role checks are per route.

`GET /` — admin, any admin role. Query `?status=` and the pair `?type=&targetId=` for a detail page's own history. Returns the full report rows plus `target_name` (the reported account's store name or name), `target_status`, and `attended_by_name`.

`GET /mine` — vendor. Only `type = 'vendor' AND target_id = <token>` rows, each with `evidence_ids` from a `GROUP_CONCAT` so the UI can tell whether a response has already been submitted.

`PATCH /:id/status` — admin + `requireAdminRole("Super Admin", "Moderator")`. Body `{ status: "resolved" | "dismissed" }`. `attended_by_user_id` is always the acting admin's own id, never accepted from the body.

`POST /` — buyer. Body `{ orderId, reason }`. Always creates `type = 'vendor'` against the order's own `vendor_id`. `404 {"error": "Order not found."}` if the order does not exist or does not belong to this buyer, looked up with `WHERE id = ? AND buyer_id = ?`. Both the reason and the reporter's display name are `escapeHtml()`'d before storage. The activity row has no actor, since a buyer filed it, not staff.

`POST /:id/evidence` — vendor. Body `{ responseText, attachmentUrls? }`, where `attachmentUrls` must be an array of strings if present. `404` collapses "no such report", "not a vendor report", and "not against you" into one response, so a vendor cannot probe for another vendor's report by id. Pure append: it does not change the report's status.

`/api/admin` (`admin.routes.js`)

`router.use(requireAuth, requireRole("admin"))` covers the whole file. Routes that need more say so.

Customers - `GET /customers` — optional `?q=` over name and email. Excludes `deleted` rows. Returns profile fields plus `avatar_url`, `last_login_ip`, and live `order_count` / `total_spent` aggregates. - `GET /customers/:id` — the same shape for one row. - `GET /customers/:id/orders` — that customer's order history with the shared item summary. - `PATCH /customers/:id/status` — Super Admin or Moderator. Body `{ status: "active" | "suspended", reason? }`. Bumps `session_version`, writes an activity row, notifies, and

emails the account. - `DELETE /customers/:id` — Super Admin. Scrubs the row to `Deleted User`, same shape as self-deletion. - `POST /customers/:id/reset-password` — any admin role. Generates a 24-byte token, stores only its SHA-256 hash with a 1-hour expiry, and emails a `reset-password.html?token=...` link to the account holder. The triggering admin never sees a credential. Writes an activity row whether or not delivery succeeds, so the attempt is auditable. - `POST /customers/:id/email` and `POST /customers/:id/email/verify` — Super Admin or Moderator. See below. - `PATCH /customers/:id/phone` — Super Admin or Moderator. Body `{ phone, reason }`, reason required. A direct edit with no OTP, because there is no SMS infrastructure in this codebase. The required reason, the activity row, and a notice email to the account are the entire friction and audit trail, which is why this is explicitly the account-recovery and fraud-support path rather than a routine one.

Admin-initiated email change. `POST /customers|vendors/:id/email` takes `{ newEmail }`, checks it is well-formed and not already used by another account of the same role (`409`), supersedes any earlier pending change, and emails a 6-digit code to the **new** address. Separately and immediately, not gated on the code ever being entered, it emails the **current** address a heads-up that a change was requested, so the real owner finds out even if the change never completes. There is no customer-facing redemption page: the account holder relays the code back through a support channel, and the admin enters it at `POST .../email/verify`, which updates the email, bumps `session_version`, and writes an activity row. Five wrong attempts or a 10-minute expiry means starting over.

Vendors - `GET /vendors` — optional `?status=` and `?kycStatus=`. Excludes `deleted`. Returns profile fields plus `avatar_url`, `last_login_ip`, live `products_count` / `orders_count` / `revenue`, `kyc_status`, `kyc_documents_submitted`, and `kyc_provider_reason` (whichever provider check failed). - `GET /vendors/:id` — one row, plus legal name parts, the full KYC detail including both provider statuses and messages, and `kyc_identity_number` decrypted for admin review. The encrypted column itself is deleted from the response object before it is sent. - `GET /vendors/:id/orders`, `GET /vendors/:id/products` — that vendor's orders and full catalogue. The product list deliberately includes `removed` listings, so the console stays an audit view rather than silently losing history. - `GET /vendors/:id/payout-account` — Super Admin or Moderator. Returns the current account with a masked number plus the `vendor_payout_account_history` rows. The full number is never returned. - `PATCH /vendors/:id/status` — Super Admin or Moderator. Body `{ status: "active" | "suspended" | "rejected", reason? }`. Moving a `pending` vendor to `active` while `vendor_verification_required` is on requires their KYC to be `verified`, otherwise `400`. Bumps `session_version`, writes an activity row, notifies, and emails. One known rough edge: the activity verb keys off the new status only, so both "pending to active" and "suspended to active" read as "Approved". - `DELETE /vendors/:id` — Super Admin. Transactional scrub: anonymises the row, delists every product, and clears `vendor_kyc.reviewed_by_user_id` references. - `DELETE /vendors/:vendorId/products/:productId` — Super Admin or Moderator. Soft-removes one listing, writes an activity row, notifies the vendor, and emails them. `400` if already removed. - `POST /vendors/:id/reset-password`, `POST /vendors/:id/email`, `POST /vendors/:id/email/verify`, `PATCH /vendors/:id/phone` — identical to the customer versions. - `PATCH /vendors/:id/kyc` — Super Admin or Moderator. Body `{ status: "verified" | "rejected", reason? }`, reason required on a rejection and re-checked here rather than trusted from the console's own modal. `404` if the vendor has no submission at all; `400` if the submission is not currently `pending` or `manual_review`. Sets `reviewed_at` and `reviewed_by_user_id`, clears `rejection_reason` on a verify, writes an activity row, notifies, and emails either outcome.

Other - `GET /users/:id/ip-history` — any admin role. Up to 100 `login_ip_history` rows for a buyer or vendor. - `GET /paystack/balance` — Super Admin. `{ balances: [{ currency, balance }], mode: "test" | "live", fetchedAt }`, balance in kobo, straight from Paystack's `GET /balance`. `503` when `PAYSTACK_SECRET_KEY` is unset, `502` with Paystack's own message when it refuses. Shown as the Paystack Balance card on the Super Admin dashboard only. - `GET /stats` — `{ totalCustomers, totalVendors, suspendedAccounts, suspendedCustomers, suspendedVendors, pendingVendors, openReports, platformOrders, platformRevenue, kycManualReview }`, every figure a real `COUNT` / `SUM` at request time, `COALESCE`'d so an empty table returns 0 rather than null. - `GET /activity` — optional `?adminId=` (Super Admin only, silently ignored otherwise since their query is already scoped) and the pair `?targetType=&targetId=`. Role-scoped server-side: a Moderator or Support admin's query is forced to `actor_user_id IS NULL OR actor_user_id = <own id>` regardless of what they send. Joins `users.name` as `actor_name`, null for system events. - `DELETE /activity/:id` and `DELETE /activity` — Super Admin. Both deliberately write no fresh activity row afterwards. Clearing the log is meant to actually clear it, not leave a new trace behind every time. - `GET /team` — any admin role. `{ id, name, email, admin_role, avatar_url }`, oldest first. - `PATCH /team/:id` — Super Admin. Body `{ adminRole }`. A no-op short-circuits to `200` without writing. `400` when the target is the only remaining Super Admin, so the console can never end up with zero. Bumps `session_version` and emails the affected admin. - `DELETE /team/:id` — Super Admin. Same last-Super-Admin guard. Runs in a transaction that first nulls `reports.reporter_user_id` / `attended_by_user_id` and `vendor_kyc.reviewed_by_user_id` for that admin, then deletes the row, so removing an admin who has reviewed things preserves the records without the foreign keys blocking the delete. Emails the removed admin. - `GET /invites` — Super Admin. Pending invites only. - `POST /invites` — Super Admin. Body `{ name, email, adminRole }`. Generates a 6-digit code, stores only its bcrypt hash with a 15-minute expiry, and emails the raw code to the invitee. - `POST /invites/:id/verify` — Super Admin. Body `{ code }`. Creates the admin row, marks the invite verified, writes an activity row, and emails a generated temporary password straight to the new admin. The temp password does not appear in the response, so the inviting admin never sees another admin's credential. - `DELETE /invites/:id` — Super Admin. Marks a pending invite `cancelled`. There is no resend route; cancel and re-invite. - `GET /settings` — any admin role, since viewing is not a management action. Returns all six booleans. - `PATCH /settings` — Super Admin. Any subset of the six, all booleans. `400` on a non-boolean value or an empty body. Returns the full updated object and writes one activity row summarising every field that changed, for example "Platform settings: disabled guest checkout, enabled maintenance mode."

`/api/site-banners` (`site-banners.routes.js`)

`GET /` — public, no auth. `[{ id, imageUrl, alt }]` ordered by `display_order`. An empty array rather than an error when none are set; the carousel falls back to its own defaults.

Everything below is `requireAuth` + `requireAdminRole("Super Admin")`, applied via `router.use()` after the public read. Changing what the storefront looks like site-wide is a platform action, not a moderation one.

- `POST /` — `{ imageUrl, alt? }`. Appends at `MAX(display_order) + 1`. Writes an activity row.
- `PATCH /:id/order` — `{ direction: "up" | "down" }`. Swaps `display_order` with the adjacent row. `400` at either end.

- `DELETE /:id` — hard delete, which is fine here because nothing foreign-keys to a banner. Writes an activity row.

`/api/notifications` (`notifications.routes.js`)

`requireAuth` only, no role check, since a notification always belongs to whoever is asking and every query is scoped to `req.user.id`.

- `GET /` — up to 100 rows, newest first.
- `GET /unread-count` — `{ count }`. Called on every page header to drive the bell badge, so it is deliberately the cheapest possible query.
- `PATCH /:id/read` — `404` if it is not this user's notification or is already read.
- `PATCH /read-all`

Rows are written by `notify()` at the point a state change happens: a new order (to the vendor), a shipment status change (to the buyer), an item marked unavailable (to the buyer), a vendor status change, a KYC submission (to every admin), a KYC decision or manual-review result (to the vendor), an account suspend or reactivate, an admin removing a listing, and a low-stock threshold crossing (to the vendor).

`/api/assistant` (`assistant.routes.js`)

`POST /chat` — `optionalAuth`, `assistantChatLimiter` (20 per 15 min). Body: `{ message, history? }`, where `history` is the prior turns passed straight through, so the frontend owns conversation state. `200`: `{ reply, matchedProducts }`. `400` if `message` is missing.

Server-side, `findCandidateProducts()` lowercases the message, strips everything but letters, digits, `₦`, and whitespace, drops words of 2 characters or fewer plus a short stopword list, then runs one `LIKE`-based query OR-ing each remaining keyword against name, description, and category, requiring `status = 'active'` on both the product and its vendor, capped at 8 rows. Those candidates are interpolated into a system prompt that instructs the model to recommend only from that list and never invent a product, price, or vendor, then sent to `claude-haiku-4-5-20251001` (overridable via `ASSISTANT_MODEL`) with `max_tokens: 400`. The reply is assembled from the response's `text` blocks.

Two honest caveats. The grounding is keyword search, not semantic search; the `description_embedding` column exists for when that is worth building. And there is no per-user cost cap beyond the IP-based rate limit, which is worth adding before real traffic, since every call is a paid API request. As noted in §1, no frontend page calls this route yet.

---

6. Security posture

What is actually enforced:

1. **Server-side role enforcement.** Every `/api/admin/*` route checks `role = 'admin'` in middleware, and the moderation and management routes layer `requireAdminRole(...)` on top. The frontend's own role checks

are a UX convenience, not the boundary.

2. **Password resets never hand a credential to an admin.** The reset link goes to the account holder; the admin sees only "Reset link sent."
3. **Admin invites are hashed and expiring.** A bcrypt-hashed 6-digit code with a 15-minute expiry, compared with `bcrypt.compare`, and the new admin's temp password is emailed to them rather than returned in the response.
4. **Rate limiting on every anonymous-reachable endpoint.** `backend/src/middleware/rateLimit.js` defines eleven per-IP limiters: signin (10/15min), admin-signin (5/15min), Google auth (10/15min), signup (20/hour), reset-password redemption (10/15min), assistant chat (20/15min), 2FA verify (20/15min), 2FA resend (5/15min), uploads (30/15min), public delivery endpoints (120/15min), and pickup/delivery code entry (20/15min). The contact form has its own (5/hour). A further one, KYC verification, is keyed per vendor account (5/hour). Separate counters, not one shared instance, so the highest-value target gets the tightest limit independently.
5. **Real client IPs behind the proxy.** `app.set("trust proxy", "loopback, linklocal, uniquelocal")`, Express's preset for "trust any number of hops through the standard private ranges, stop at the first public address". On cPanel the only hop is Apache/Passenger on the same machine (loopback). The preset replaced a hardcoded `1`, which on Render stopped one hop too early: every row in `login_ip_history` recorded a private `10.x.x.x` address, and the rate limiters saw one shared IP for every visitor. A real client's own public IP can never fall in those ranges, and the edge proxy overwrites whatever `X-Forwarded-For` a client sent, so nothing after the edge is attacker-controlled.
6. **Stored XSS closed at write time.** `activity_log.message`, `notifications.message`, `reports.reason`, and `report_evidence.response_text` are all rendered with `innerHTML` by frontend JS with no escaping of their own. Vendor store names, product names, buyer names, report reasons, and admin-typed reasons are `escapeHtml()`'d (`backend/src/utils/escapeHtml.js`) immediately before they are written, so every current and future consumer of the stored value is safe without having to remember. One field is deliberately escaped at *render* time instead: `actor_name` in the activity feed is a live `JOIN` onto `users.name`, computed fresh on each read, so there is no write-time moment to escape it at; `admin/assets/ui.js` handles it.
7. **Audit log integrity.** Every `activity_log` row is written server-side inside the route that performed the mutation. Nothing is client-supplied.
8. **Secrets at rest.** Vendor NUBANs and KYC identity numbers are AES-256-GCM encrypted with a key derived by SHA-256 from `ENCRYPTION_KEY`, so the env value can be any non-empty string rather than strictly 32 hex bytes, which matters because the key was first generated by Render's `generateValue: true`, which does not guarantee hex. The same value moved to cPanel's `.env` unchanged; a different key would make every stored number unreadable. Passwords are bcrypt. Reset tokens, 2FA codes, and email-change codes are SHA-256 hashed. Invite codes are bcrypt hashed.
9. **Session revocation.** `session_version` plus the per-request account re-read means a suspension, password change, role change, or email change takes effect on the next request rather than on token expiry.
10. **KYC documents are not public URLs.** Cloudinary `type: authenticated` with signed URLs, and `POST /me/kyc` validates that the submitted URLs are genuinely Cloudinary authenticated paths.

11. **CORS never fails open.** `CORS_ORIGINS` is a comma-separated allow-list, and an unset value falls back to the known production frontend rather than `*`.

What is still weak, stated plainly:

- **General request-body validation is patchy.** Specific routes validate specific fields carefully (quantities, prices, keyword counts, NUBAN format, CAC format, review length, state values), but there is no systematic type/length validation layer across arbitrary fields.
- **No automated tests.** See `backend/README.md`.
- **No per-user cost cap on the assistant.**
- **No pagination.** Every list route caps at 200 rows, which is a safety net rather than a real answer once any table gets large.

---

7. Integrations, and how each degrades

Each of these is reached through an env var read at startup, and each fails in a specific, deliberate way when it is not configured.

Integration	Module	Behaviour when unconfigured
Cloudinary	`utils/cloudinary.js`	Uploads fail with a real error. Nothing else breaks.
cPanel SMTP	`utils/mailer.js`	`sendEmail()` logs `[email:not-configured]` with the code, link, or password inline and returns `{ delivered: false }`. Every flow that emails something still completes.
Paystack	`utils/paystack.js`	Every call throws with `status: 503` when `PAYSTACK_SECRET_KEY` is unset: checkout, payout-account setup and the balance card stop working; payouts and refunds already recorded stay `failed` / `queued` until it's back. Paystack-side failures throw 422 or 502 with Paystack's own message. Checkout, transfers and refunds all carry a VETRA-generated reference stored before the call, so a retry can't charge or pay twice.
IDCheck.ng	`utils/idcheck.js`	Throws with `statusCode: 503` and "Identity verification is not configured yet." when `IDCHECK_API_KEY` is unset. Every provider-side failure (record not found, rejected key, empty wallet, upstream down, a 20-second timeout) throws 422 or 502 with a readable message, which the KYC route records as a failed check and sends to manual review.
Anthropic	`assistant.routes.js`	The assistant chat errors. Nothing else is affected.
Google	`utils/googleAuth.js`	"Continue with Google" fails; email/password auth is unaffected.

`idcheck.js` details worth preserving: IDCheck's public GitBook docs (`docs.idcheck.ng`) describe an older API with a different auth header and paths; the client follows the live OpenAPI spec at `https://api.idcheck.ng/v1/openapi.json`. Every call sends `Authorization: Bearer <key>` and a fresh `Idempotency-Key`, which the API requires on every POST. Raw provider payloads are never returned or stored.

`mailer.js` details worth preserving: `sendEmail()` never throws (it runs after the real action succeeded), it checks `info.rejected` because an SMTP server can accept the connection yet reject the recipient, and its connect/socket timeouts are short (10–20s) because every send is awaited inside a request handler. `verifyMailer()` runs at startup so a wrong password shows in the log immediately.

`logActivity()`, `notify()`, and `sendEmail()` are all best-effort by design: they run after their call site's real primary write, and a failure is logged rather than thrown, because turning an already-successful action into a 500 invites a client retry that duplicates the action.

---

8. What is genuinely not built

These are honest gaps, not things awaiting a wiring pass.

- **Automated recovery of a refund made after the vendor was paid.** A manual refund is refused once escrow has released; recovering that money from the vendor happens outside VETRA.
- **Third-party courier integrations.** VETRA's own riders are fully built (`DELIVERY_SYSTEM.md`), including escrow auto-release and live rider location. A vendor who chooses "I'll ship it myself" still types a carrier name and tracking number by hand; nothing calls GIG, DHL or similar APIs.
- **Buyer to vendor chat.** No schema, no routes, no real-time layer. The frontend feature is hidden site-wide, and the old UI-only mock (`customer/chat.html`) was deleted, so there is nothing to wire a backend to. Smartsupp's third-party widget on the customer and vendor apps is support chat with VETRA, not messaging between a buyer and a vendor.
- **Semantic product search.** The assistant grounds on keyword `LIKE` matching. `products.description_embedding` is provisioned and unwritten.
- **SMS.** Nothing in the product currently needs it, which is exactly why the admin-initiated phone change has no OTP.
- **Automated tests.** Nothing beyond manual verification.
- **Pagination.** Every list route has a hard cap instead.

---

9. What not to over-build

- Do not build a permissions system finer-grained than the three admin roles. The UI has no surface for anything more granular.
- Do not build real-time infrastructure for the admin console. Nothing there needs to push to an open tab.

- Do not build multi-currency or multi-region support. The whole product is NGN and Nigeria specific: phone formats, state list, Paystack, IDCheck.ng.
- Do not stand up a dedicated vector database for product search. Application-level similarity over embeddings stored in MySQL handles a marketplace catalogue of this size, and a managed vector service is real infrastructure to pay for and operate.
- Do not denormalise `sales_count`, `rating`, or `review_count` into stored columns yet. They are live aggregates today, correct by construction, and the join cost is not measurable at this volume. Cache them when list latency is a real observed problem, not before.

---

10. Scaling, when a real constraint is hit

Signals rather than traffic numbers: `ER_USER_LIMIT_REACHED` or entry-process faults under normal load, list endpoints slowing down because 200-row caps are being hit routinely, outgoing email hitting the plan's hourly cap, or wanting MySQL tuning shared hosting will not give you.

Likely order of moves, cheapest first:

1. **Pagination** on the list routes, before anything infrastructural. The `LIMIT 200` caps are a known correctness liability well ahead of being a performance one.
2. **A bigger cPanel plan** with a higher Entry Processes cap, at which point `DB_CONNECTION_LIMIT` in `~/vetra-api/.env` can rise from 10. Check the new plan's actual cap first rather than trusting any number written in this repo, and leave headroom, since the Node app and the cron job share the same budget.
3. **A job queue**, so notifications, emails, and Cloudinary uploads move off the request path. The delivery sweep already runs outside requests as a cron job, but everything else still happens inline.
4. **A VPS or container platform with root**, only if WebSocket chat becomes a real requirement. That is the move that lifts the no-persistent-connections constraint, and it brings real new responsibilities with it: backups, OS and MySQL patching, tuning, and monitoring that a managed platform currently handles.

Nothing in application code changes for a database move. `backend/src/db.js` reads `DB_HOST` / `DB_PORT` / `DB_USER` / `DB_PASSWORD` / `DB_NAME` / `DB_SSL` from the environment. Migrate the data, verify row counts table by table against the old database rather than trusting a clean exit code, point the env vars at the new host, and keep the old database readable for a rollback window.